

תוכן עניינים

מבוא.....	4
מטרות התוכנה.....	4
קהל היעד.....	5
טכנולוגיה.....	5
תיאור המשחק / לומדה:.....	6
התאמת מימדי לוח אישית:.....	6
עמודי תפריט:.....	6
חוקי המשחק / לומדה:.....	6
כניסה לאפליקציה:.....	6
Database מבנה הנתונים.....	7
קשרי הגומלין בין הטבלאות:.....	7
Database:טבלאות ב-.....	7
קוד עבור מסד הנתונים.....	11
מפת האפליקציה.....	13
תיאור מסכים.....	14
תיאור הקבצים.....	14
הקבצים הראשיים בסודוקו:.....	16
נכסי פרויקט.....	27
קבצי משחק.....	27
עמודים.....	78
מחלקות סטטיות.....	78
דפים.....	79
דף בית.....	79
דף יצירת משחק.....	82
דף משחק.....	89
דף הסטוריה ודף משחקים שמורים.....	104
דף הצגת הוראות.....	118
עמוד הוראות לדוגמה.....	122
דף הגדרות תכנה.....	125
אלגוריתמים ויכולות מיוחדות.....	130
יצירת לוח פתור.....	130
Color themes.....	131

חזרה על קוד עיצובי.....	132
רספונסיביות ודינמיות מסך.....	132
רכיבים חיצוניים.....	133
רפלקציה.....	134
בעיות במהלך הדרך	134
דברים שלמדתי	134
תהליך עבודה.....	134
תוספות אפשריות	135

מבוא

מטרות התוכנה

שיפור יכולת החשיבה:

פתרון סודוקו מפתח את החשיבה הלוגית ואת מיומנויות פתרון הבעיות. התכנה מאפשרת למשתמשים לתרגל ולשפר את כישוריהם האנליטיים והלוגיים, מה שיכול להועיל בתחומים רבים אחרים בחיים.

שדרוג חווית פתירת הסודוקו מפתירה על נייר על נייר :
אפליקציית הסודוקו מאפשרת אמצאים ויזואליים העוזרים למשתמש ומשפרים את נוחות פתירת הסודוקו מבלי להגביל את היצירתיות ואת קושי הפאזל. כגון: סימוני תאים באותו טור, שורה או קופסא וסימוני תאים עם טקסט זהה.

חידוש ביחס לאפליקציות אחרות :

אפליקציית הסודוקו שלי מאפשרת יכולות שאפליקציות סודוקו הקיימות בשוק לא מאפשרות, כמו הגדלת כמות התאים בלוח. בנוסף, האפליקציה שלי נותנת לשחקן את החופש לפתור הבעיה בעצמו, כמו חידה שאף אחד לא פתר לפניו (מזכיר פתרון על דף) לעומת אפליקציות רבות המגבילות את המשתמש מלמצוא פתרון משלו ובכך מגבילות את היצירתיות.

בידור ופנאי:

תכנת הסודוקו מעוצבת בצורה המספקת חוויה מהנה ומאתגרת למשתמש. היא משמשת כאמצעי להעברת זמן באופן מהנה, עם מגוון רחב של רמות קושי המתאימות למשתמשים מתחילים ומתקדמים כאחד.

פיתוח אפליקציה מסחרית :

תכנה זו יכולה להיות הבסיס לאפליקציה מסחרית לציבור הרחב הכוללת פונקציות נוספות כמו תחרויות, דירוגים ורשתות חברתיות.

שיפור יכולות התכנות:

פיתוח תכנת סודוקו דורש ידע נרחב בתכנות, כולל תכנון והקמת ממשק משתמש (UI design), תכנות מונחה עצמים (OOP), מבני נתונים, ושימוש במסדי נתונים. הפרויקט יכול לשמש כתירגול מצוין לכתיבת קוד יעיל, קריא ונקי, ולשיפור המיומנויות הטכניות של המפתחים.

יצירת כלים חינוכיים:

תכנת הסודוקו יכולה לשמש ככלי חינוכי ללימוד מתמטיקה, לוגיקה וחשיבה ביקורתית בבתי ספר ובמסגרות לימוד אחרות. היא מספקת תרגול מעשי בניתוח בעיות ופתרון, ועוזרת לתלמידים לפתח כישורים חשובים בצורה מהנה ומעשירה.

השגת מטרות אלו דורשת השקעה בממשק משתמש ידידותי, בבנייה של לוחות סודקו מאתגרים ומגוונים, ובהוספת תכונות מתקדמות המעשירות את חווית המשתמש ומגדילות את הערך החינוכי והתפעולי של התכנה.

קהל היעד

קהל היעד של התכנה הוא אנשים הנהנים ממשחקי חשיבה ומחפשים דרך נוחה ופשוטה להפעיל את המוח במהלך היום.

כמו כן, קהל היעד הוא אנשים בכל גיל המאפשר להם שימוש בתכנה - בן דודי בין ה 5 נהנה לשחק בתכנה ואחרי שלימדתי אותו את החוקים, פתר לוחות של 2*2 ברמה בינונית. כמו כן, גם סבתי נהנתה מהתכנה והסתגלה למעבר מפתירה על נייר לפתירה במחשב.

טכנולוגיה

השתמשתי בטכנולוגיה של WPF לפיתוח האפליקציה שלי, שמספקת עיצוב עשיר ויכולות גרפיות מתקדמות בסביבת .NET. ב-WPF הממשק המשתמש מוגדר באמצעות קבצי XAML (Extensible Application Markup Language), שמאפשרים לי לבנות ולערוך את ה-UI של היישום בצורה ברורה ומבלי לדרוש ידע עמוק בתכנות.

הטכנולוגיה מאפשרת לי ליצור אנימציות ואובייקטים גרפיים מתקדמים, שמשפרים את חוויית המשתמש ומעניקים מראה ותחושה מודרניים ליישום. כמו כן WPF, מספקת תמיכה מובנית בניהול קשרי נתונים, שמאפשרת לי לקשר ולהציג מידע ממקורות שונים כגון מסדי נתונים וקבצי XML בצורה יעילה ונוחה לתחזוקה.

בנוסף, בתכנות מונחה אירועים (Event-Driven Programming) הפעולות ביישום מופעלות בתגובה לאירועים שונים שקורים בממשק המשתמש. כלומר, במקום להריץ קוד בצורה רציפה, היישום יגיב לאירועים כגון לחיצת משתמש על כפתור, שינוי בערך בתיבת טקסט, או כל אירוע אחר שמופעל במהלך השימוש ביישום. זה מאפשר לי לתכנן את היישום כך שהתגובה לפעולות המשתמש תהיה מהירה ויעילה.

השימוש ב-WPF בשילוב עם תכנות מונחה אירועים איפשר לי ליצור UI מודרני ואפקטיבי בצורה יעילה ומותאמת אישית.

תיאור המשחק / לומדה:

משחק הסודוקו שלי הוא אפליקציה מקיפה שמטרתה לספק חוויית סודוקו עשירה ומותאמת אישית לשחקנים בכל הרמות. המשחק כולל מספר עמודים עם הוראות מפורטות, מצבי משחק שונים ותכונות ידידותיות למשתמש המשפרות את חוויית המשחק.

התאמת מימדי לוח אישית:

שחקנים יכולים לבחור את רוחב וגובה המשבצות הגדולות, מה שמאפשר מגוון גדלי רשת וקונפיגורציות. לדוגמה, ניתן לקבל לוח 4×4 עם קופסאות 2×2 , לוח 6×6 עם קופסאות 2×3 , או כל שילוב אחר שבו גודל הלוח הכולל הוא מכפלת מימדי הקופסא.

עמודי תפריט:

עמוד המשחק: המקום שבו השחקנים יכולים להתחיל משחק חדש עם מימדי רשת ורמות קושי מותאמים אישית. העמוד כולל תכונות כמו רמזים, ביטול וניהול הערות כדי לסייע לשחקנים.

עמוד השמורים: מציג את כל המשחקים השמורים, ומאפשר לשחקנים להמשיך כל משחק מהנקודה בה הפסיקו.

עמוד ההיסטוריה: מציג לוג של המשחקים שהושלמו, כולל ניקוד, זמני השלמה ורמות קושי.

עמוד ההגדרות: מאפשר התאמה אישית של מראה והתנהגות האפליקציה, כמו הפעלת וכיבוי צלילים ומוזיקה, ושינוי ערכות נושא.

משוב קולי

המשחק מספק משוב קולי עבור פעולות שונות. לדוגמה, השמעת צליל כאשר מתבצעת הזנה חוקית או צליל שונה כאשר הזנה לא חוקית חורגת מהמכפלה המקסימלית המותרת של מימדי הרשת.

חוקי המשחק / לומדה:

כל שורה: חייבת להכיל את כל הספרות (או האותיות) מהטווח המוגדר, ללא חזרות.

כל עמודה: חייבת להכיל את כל הספרות (או האותיות) מהטווח המוגדר, ללא חזרות.

כל קופסא: חייבת להכיל את כל הספרות (או האותיות) מהטווח המוגדר, ללא חזרות.

כניסה לאפליקציה:

ניתן לראות מסך כניסה עם תמונה ב DOUBLE ANIMATION של סודוקו. (ראה עמוד פתיחה בדפי הפרוייקט)

הניווט מתבצע באמצעות תפריט מבוסס על RADIO BUTTON דרך MainWindow.

כדי לסגור את האפליקציה או לשנות את גודל והופעתו של המסך יצרתתי 3 כפתורים בצידו העליון מימין.

מבנה הנתונים Database

קשרי הגומלין בין הטבלאות:

לא היה צורך בקשרי גומלין בין טבלאות בעבודתי

טבלאות ב-Database:

tbl_games:

בטבלה זו מאוכסנים הנתונים המתארים משחק.

שם שדה	סוג נתונים	תיאור (אופציונלי)
Id	מספור אוטומטי	
Current	כן/לא	was the game running when app was closed
Solved	כן/לא	Was the game solved
Time	טקסט קצר	The time it took to solve or end the game
BoardCode	טקסט ארוך	An encrypted code containing the value of each cell and notes
PuzzleCode	טקסט ארוך	An encrypted code containing the solved value of each cell and if it was initial
gameName	טקסט קצר	Name of game given by the user
HintsTaken	מספר	Hints taken in the game till end
ChecksTaken	מספר	Check boards taken in the game till the end
BoxHeight	מספר	Height of boxes
BoxWidth	מספר	Width of boxes
GameDate	טקסט קצר	Date of game
DifficultyLevel	טקסט קצר	Difficulty level of game

tbl_settings:

בטבלה זו מאוכסנים נתונים על ההגדרות של המשתמש.

שם שדה	סוג נתונים	
Id	מספור אוטומטי	
SoundOn	כן/לא	
Theme	טקסט קצר	Mame of color theme
MusicOn	כן/לא	
MarkRelated	כן/לא	
MarkSameText	כן/לא	
AllowNotes	כן/לא	

שימוש במסד הנתונים בתכנה

בתחילת ריצת התכנית, התכנה עוברת על כל השורות בטבלת המשחקים במסד הנתונים ויוצרת לכל שורה אובייקט GameInfo כדי להכיל את המידע.

בהתאם להאם המשחק פתור, היא ממינת את ה אובייקטים לרשימה של המשחקים השמורים ולרשימה של המשחקים בהסטוריה:

```
/// <summary>
/// Sets the saved games pages from the database.
/// </summary>
1 reference
private void SetSavedGamesFromDB()
{
    historyPage = new SaverPage(true); // Initialize history page
    savedPage = new SaverPage(false); // Initialize saved games page

    DataTable dt = DBHelper.GetDataTable("SELECT * FROM tbl_games"); // Retrieve games data from database

    foreach (DataRow dr in dt.Rows)
    {
        // Create GameInfo object from database row
        GameInfo gameInfo = new GameInfo(
            Convert.ToInt32(dr["Id"]),
            dr["GameName"].ToString(),
            dr["BoardCode"].ToString(),
            dr["PuzzleCode"].ToString(),
            dr["Time"].ToString(),
            Convert.ToInt32(dr["HintsTaken"]),
            Convert.ToInt32(dr["ChecksTaken"]),
            dr["GameDate"].ToString(),
            (DifficultyLevel)Enum.Parse(typeof(DifficultyLevel), dr["DifficultyLevel"].ToString()),
            Convert.ToBoolean(dr["Solved"]),
            Convert.ToBoolean(dr["Current"]),
            Convert.ToInt32(dr["BoxHeight"]),
            Convert.ToInt32(dr["BoxWidth"])
        );

        // Determine where to add the game info based on solved and current flags
        if (gameInfo.Solved)
        {
            historyPage.AddItemToList(gameInfo); // Add to history page
        }
        else
        {
            if (gameInfo.Current)
            {
                SaverPage.DeleteGameFromDB(gameInfo); // Remove from database if current game
                gamePage = new GamePage(gameInfo); // Initialize game page
                return; // Exit method after initializing game page
            }
            else
            {
                savedPage.AddItemToList(gameInfo); // Add to saved games page
            }
        }
    }
}
```

מתוך MainWindow.xaml.cs

כמו כן, בתחילה גם ההגדרות נקבעות להגדרות השמורות במסד הנתונים בטבלה tbl_settings :

מתוך MainWindow.xaml.cs

```
/// <summary>
/// Sets the application settings from the database.
/// </summary>
1 reference
private void SetSettingsFromDB()
{
    DataTable dt = DBHelper.GetDataTable("SELECT * FROM tbl_settings");
    DataRow dr = dt.Rows[0]; // Get the first row (assuming single settings row)

    // Load settings values from database
    Settings.markSameText = Convert.ToBoolean(dr[DBConstants.Settings_Parameters.SameText]);
    Settings.markRelated = Convert.ToBoolean(dr[DBConstants.Settings_Parameters.MarkRelated]);
    Settings.soundOn = Convert.ToBoolean(dr[DBConstants.Settings_Parameters.SoundOn]);
    Settings.musicOn = Convert.ToBoolean(dr[DBConstants.Settings_Parameters.MusicOn]);

    // Set theme if valid theme found in database
    if (Enum.TryParse(dr[DBConstants.Settings_Parameters.Theme].ToString(),
        out ColorThemes theme))
    {
        Settings.theme = theme;
        ThemeControl.SetColors(Settings.theme); // Apply theme colors
    }
}
```

בעת סגירת תכנה, אם משחק רץ, התכנה מוסיפה אותו למסד הנתונים:

מתוך MainWindow.xaml.cs

```
/// <summary>
/// Closes the application window and performs necessary cleanup.
/// </summary>
1 reference
private void CloseApp_Click(object sender, RoutedEventArgs e)
{
    // Stop music if playing
    if (Settings.musicOn)
    {
        SoundPlayer.StopMusic();
    }

    // Insert current game into database if game page exists
    if (gamePage != null)
    {
        SaverPage.InsertGame(gamePage.GetGameInfo(false, true));
    }

    UpdateSettingsInDB(); // Update settings in database
    Application.Current.Shutdown(); // Shutdown the application
}
```


בעת סיום משחק, התכנה מוסיפה שורה לרשימות בזיכרון התכנה ולמסד הנתונים עם המידע על המשחק:

```
/// <summary>
/// Ends the current Sudoku game session.
/// </summary>
/// <param name="isSolved">Indicates whether the game is solved.</param>
/// <param name="toSave">Indicates whether to save the game.</param>
7 references
public void End(bool isSolved, bool toSave)
{
    this.timer.Stop();
    this.inProgress = false;

    var window = (MainWindow)Application.Current.MainWindow;

    if (window.gamePage != null) // wasn't already ended
    {
        if (toSave)
        {
            GameInfo gameInfo = window.gamePage.GetGameInfo(isSolved, false);
            if (isSolved)
            {
                window.historyPage.AddItemToListAndDB(gameInfo);
            }
            else
            {
                window.savedPage.AddItemToListAndDB(gameInfo);
            }
        }

        window.gamePage.DisableGameControls();
        window.gamePage = null;
    }
}
```

מתוך Game.cs

בסגירה התכנה גם מעדכנת את ההגדרות

```
public void UpdateSettingsInDB()
{
    string sqlStmt = DBConstants.UpdateSettingsQuery; // SQL statement to update settings

    // Define parameters for the SQL statement
    OleDbParameter[] parameters =
    {
        new OleDbParameter(DBConstants.AT + DBConstants.Settings_Parameters.SameText, Settings.markSameText),
        new OleDbParameter(DBConstants.AT + DBConstants.Settings_Parameters.MarkRelated, Settings.markRelated),
        new OleDbParameter(DBConstants.AT + DBConstants.Settings_Parameters.SoundOn, Settings.soundOn),
        new OleDbParameter(DBConstants.AT + DBConstants.Settings_Parameters.MusicOn, Settings.musicOn),
        new OleDbParameter(DBConstants.AT + DBConstants.Settings_Parameters.Theme, Settings.theme.ToString()),
        new OleDbParameter(DBConstants.AT + DBConstants.Settings_Parameters.AllowNotes, Settings.allowNotes)
    };

    DBHelper.ExecuteCommand(sqlStmt, parameters); // Execute the database command with parameters
}
```

קוד עבור מסד הנתונים

```
using System;
using System.Data;
using System.Data.OleDb;

namespace DAL
{
    public class DBHelper
    {
        /// <summary>
        /// Creates and returns a new OleDbConnection object.
        /// </summary>
        /// <returns>An OleDbConnection object with the connection string set.</returns>
        public static OleDbConnection GetConnection()
        {
            string connString = @"Provider=Microsoft.ACE.OLEDB.12.0;
                                Data
Source=C:\Users\Cyber_User\source\repos\Sudoku_WPF\DAL\Data\Sudoku_DB1.accdb;
                                Persist Security Info=True";
            OleDbConnection conn = new OleDbConnection(connString);
            return conn;
        }

        /// <summary>
        /// Creates and returns a new OleDbCommand object.
        /// </summary>
        /// <param name="conn">The OleDbConnection to use for the command.</param>
        /// <param name="sqlstmt">The SQL statement to execute.</param>
        /// <returns>An OleDbCommand object with the specified SQL statement and
connection.</returns>
        public static OleDbCommand GetCommand(OleDbConnection conn, string sqlstmt)
        {
            OleDbCommand cmd = new OleDbCommand(sqlstmt, conn);
            return cmd;
        }

        /// <summary>
        /// Executes the provided SQL statement and returns the results as a DataTable.
        /// </summary>
        /// <param name="sqlstmt">The SQL statement to execute.</param>
        /// <returns>A DataTable containing the results of the SQL query.</returns>
        public static DataTable GetDataTable(string sqlstmt)
        {
            OleDbConnection conn = GetConnection();
            OleDbCommand cmd = GetCommand(conn, sqlstmt);
            OleDbDataAdapter adp = new OleDbDataAdapter();
            adp.SelectCommand = cmd;
            DataTable dt = new DataTable();
            adp.Fill(dt);
            return dt;
        }

        /// <summary>
        /// Executes a non-query SQL statement with the provided parameters.
        /// </summary>
        /// <param name="sqlStmt">The SQL statement to execute.</param>
        /// <param name="parameters">The parameters to add to the SQL command.</param>
        public static void ExecuteCommand(string sqlStmt, params OleDbParameter[]
parameters)
        {

```

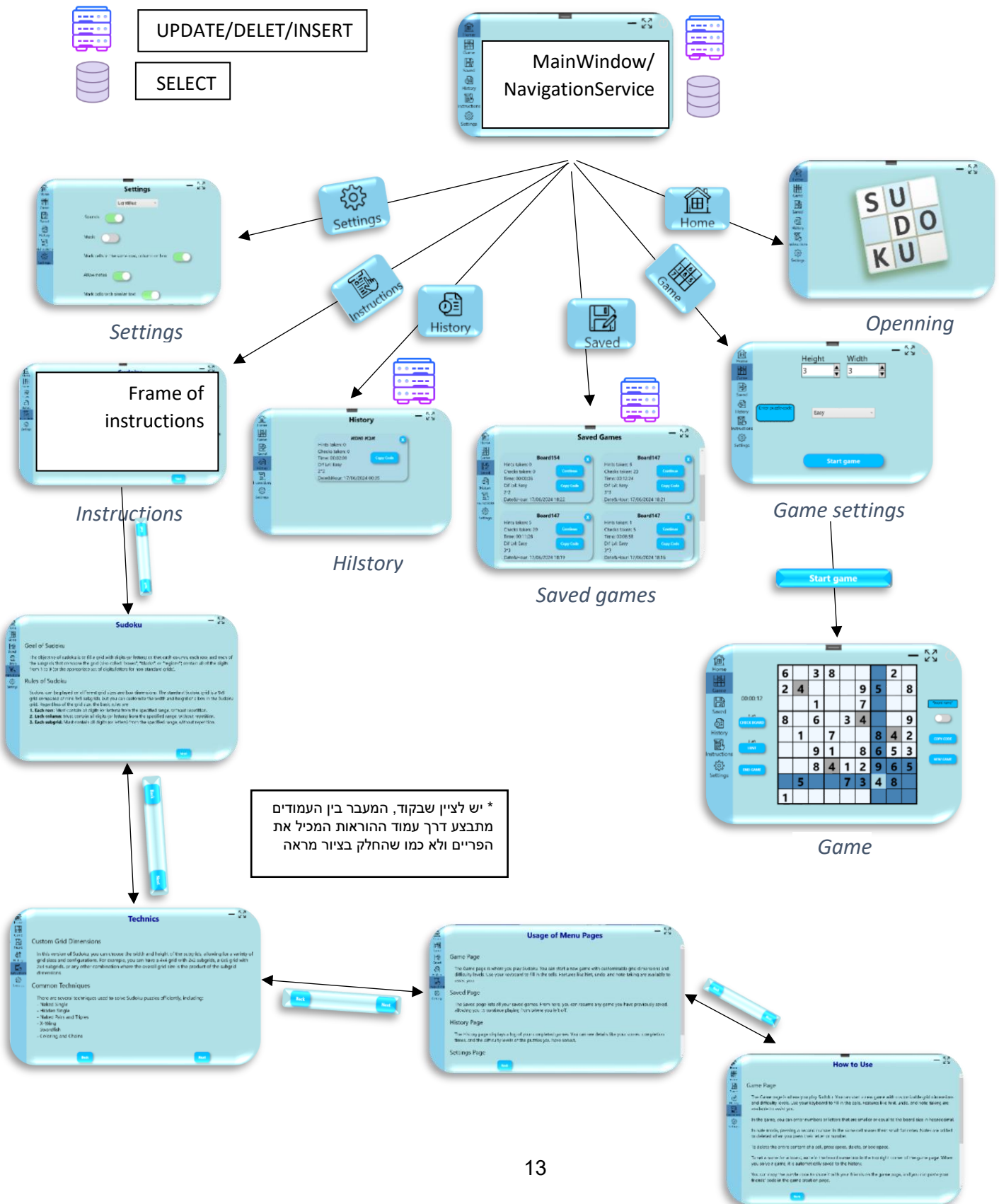
```

        OleDbConnection conn = GetConnection();
        try
        {
            conn.Open();
            OleDbCommand cmd = new OleDbCommand(sqlStmt, conn);
            cmd.Parameters.AddRange(parameters);
            int ret = cmd.ExecuteNonQuery();
        }
        finally
        {
            conn.Close();
        }
    }

    /// <summary>
    /// Retrieves the next available ID for a specified table and column.
    /// </summary>
    /// <param name="tableName">The name of the table to query.</param>
    /// <param name="idColumnName">The name of the ID column to query.</param>
    /// <returns>The next available ID as an integer.</returns>
    public static int GetNextId(string tableName, string idColumnName)
    {
        int nextId = 1; // Default starting point for IDs
        using (OleDbConnection conn = GetConnection())
        {
            string query = $"SELECT MAX([{idColumnName}]) FROM [{tableName}]";
            using (OleDbCommand cmd = new OleDbCommand(query, conn))
            {
                conn.Open();
                var result = cmd.ExecuteScalar();
                if (result != null && result != DBNull.Value)
                {
                    nextId = Convert.ToInt32(result) + 1;
                }
            }
        }
        return nextId;
    }
}

```

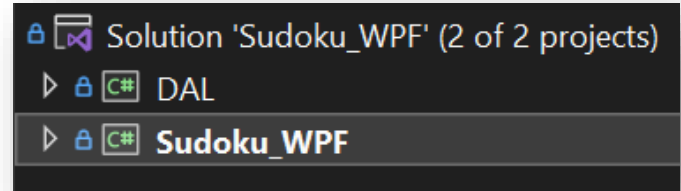
מפת האפליקציה



תיאור מסכים

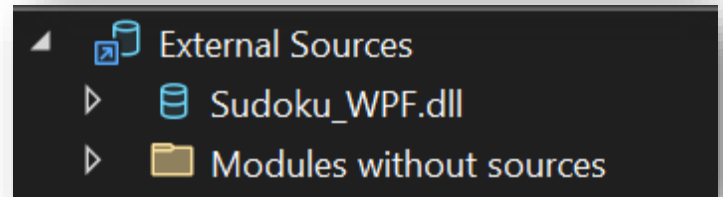
תיאור הקבצים

קובץ הsolution מכיל 2 צמתים(nodes):
DAL לטיפול במסד הנתונים והפרויקט סודוקו
עצמו.



במצב דיבאג גם מתאפשרת הצפייה בצומת
"מקורות חיצוניים":

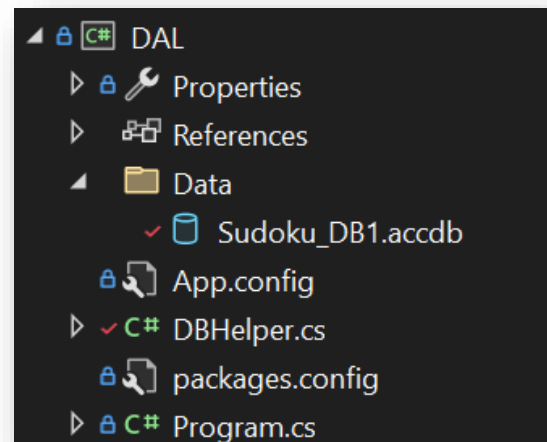
במקורות החיצוניים נמצאות הספריות
החיצוניות כמו dll אשר משומשות בפרויקט
ומקורן לא כלול ישירות בפרויקט



הפרויקט DAL (Data Access Layer) מטפל
בגישה הנתונים המושמש בפרויקט.

בסיס הנתונים המושמש הוא "Sudoku_DB1"
הנמצא בתיקיית "Data" (ראה "[מבנה מסד הנתונים](#)").

DBHelper מכיל מחלקה המכילה את השיטות
שעוזרות לגשת ולערוך מידע מבסיס הנתונים
(ראה "[קוד לבסיס הנתונים](#)").



בנוסף, קיימות גם תיקיות סטנדרטיות שנוצרות אוטומטית כחלק מהפרויקט ב-Visual Studio:

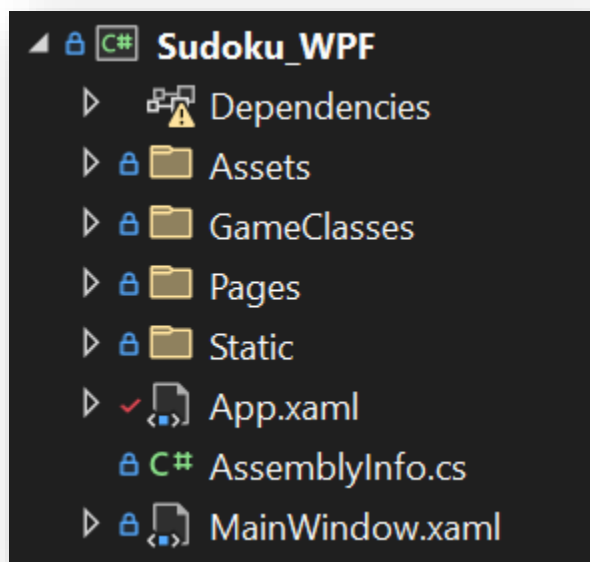
packages.config: קובץ קונפיגורציה של NuGet ([ראה רכיבים חיצוניים הסבר](#)).

References: תיקייה עם ההפניות לספריות וקבצים חיצוניים שהפרויקט תלוי בהם.

Program.cs: קובץ הקוד הראשי של הפרויקט, לא מפעיל דבר כיוון שפונקציות ב DBHelper נועדו להיות סטטיות

Properties: מכילה קבצים שמגדירים מאפיינים של הפרויקט, כמו AssemblyInfo.cs, שבו נשמר מידע על ההרכבה (assembly).

App.config: קובץ קונפיגורציה של האפליקציה שמכיל הגדרות כמו חיבורי מסד נתונים, פרמטרים של אפליקציה ועוד.



קובץ הפרויקט מכיל תיקיות רבות המסדרות את קבצי הפרויקט שיצרתי לפי תפקיד (עמוד, מחלקת משחק...) וסוג קובץ (PNG למשל) בצורה אינטואיטיבית, נוחה לשימוש ודיבוג ומסודרת:

Assets: תיקיה המכילה את כל הנכסים של הפרויקט: תמונות, אייקונים, צלילים, מסיקות רקע, סטיילים ומצבי צבעים.

GameClasses: תיקיה המכילה קבצי cs המחזיקים במחלקות התורמות ליצירת ותפקוד משחק סודוקו בודד.

Pages: תיקיה המכילה את העמודים המשומשים במשחק.

Static: תיקיה המכילה את המחלקות הסטטיות שמטרתן להחזיק מידע מוכר לכולם במהלך ריצת התכנית.

- ראה הסבר מפורט על כל אחת מתיקיות אלו בהמשך.

נכסי פרויקט

התיקיה מכילה את כל הנכסים של הפרויקט מסודרים בתיקיות:

תמונות - PNG

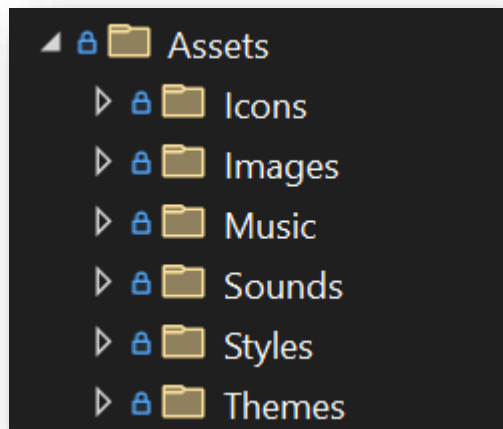
אייקונים - PNG

צלילים - WAV

מוסיקות רקע - WAV

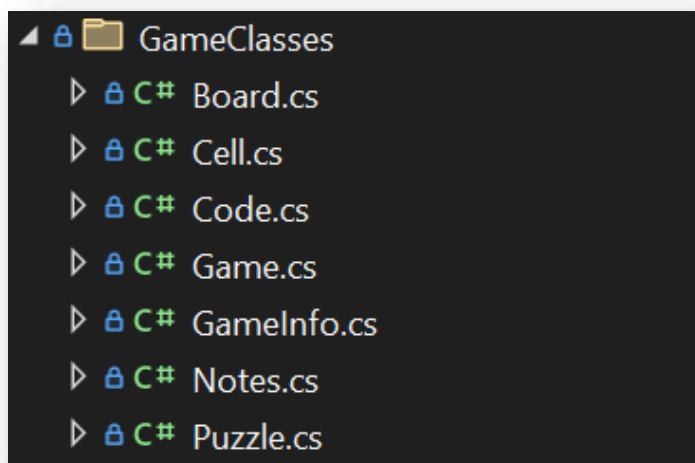
סטיילים - XAML (resource dictionary)

נושאי צבעים - XAML (resource dictionary)



קבצי משחק

התיקיה מכילה קבצי CS המחזיקים במחלקות התורמות ליצירת ותפקוד משחק סודוקו בודד.



מחלקת לוח מחזיקה באובייקט Puzzle.

תפקידה העיקרי של המחלקה הוא לנהל את לוח המשחק של הסודוקו, הן מבחינת לוגיקת המשחק והן מבחינת הממשק הגרפי. תפקידי המחלקה המרכזיים:

יצירת לוח המשחק: המחלקה יוצרת את ה-Grid-המכיל את תאי הסודוקו, מוסיפה שורות, עמודות ומסגרות (Borders) ומציבה את התאים במקומות הנכונים.

אתחול הלוח: המחלקה מאתחלת את הלוח עם הנתונים של הפאזל, מציבה את הערכים ההתחלתיים בתאים ומגדירה אילו תאים ניתנים לעריכה ואילו תאים קבועים.

ניהול לוגיקת המשחק: המחלקה מטפלת באירועים של שינוי ערכים בתאים, בדיקת פתרון הלוח, הצגת האנימציה של פתרון הפאזל והפעלת אירועים כאשר המשחק מסתיים.

הצגת מצב הלוח: המחלקה בודקת את מצב הלוח ומציגה שינויים ויזואליים לפי הערכים שהוזנו בתאים, למשל, שינוי צבעי הרקע והטקסט בהתאם לתקינות הנתונים.

הצגת הפתרון: המחלקה מאפשרת הצגת הפתרון המלא של הפאזל.

אינטראקציה עם המשתמש: המחלקה מספקת פונקציות לניהול הפוקוס על תאים, קבלת התא שנמצא בפוקוס והתמודדות עם שינוי גודל המסגרות.

המחלקה כוללת גם משתנים סטטיים ואירועים המאפשרים לנהל את מצב הלוח ולהגיב לפעולות המשתמש.

קוד מחלקת Board :

```
using System;  
using System.Collections.Generic;  
using System.Windows;  
using System.Windows.Controls;  
using System.Windows.Media;
```


המחלקה Code מספקת פעולות להצפנה ופענוח של מחרוזות באמצעות ממשק ה-Windows Data Protection API.

שיטת הצפנה (Protect):

שיטה זו מקבלת מחרוזת כקלט ומצפינה אותה באמצעות API(Application Interface) של Windows Data Protection.

הקלט ממומר למערך של bytes באמצעות קידוד ASCII.

Entropy (נתוני כניסה לצורך הצפנה) מוגדרים באופן דינמי באמצעות השימוש במידע מזהה של האסמבלי הנוכחית.

תוצאת ההצפנה היא מחרוזת בפורמט Base64 המייצגת את הנתונים המוצפנים.

קוד מחלקת code :

```
using System;
using System.Security.Cryptography;
using System.Text;
using System.Reflection; // for Assembly

namespace Sudoku_WPF.GameClasses
{
    /// <summary>
    /// Provides methods to protect and unprotect data using the Windows data protection
    API.
    /// </summary>
    internal class Code
    {
        /// <summary>
        /// Protects a string using Windows data protection API.
        /// </summary>
        /// <param name="str">The string to be protected.</param>
        /// <returns>A base64-encoded string representing the protected data.</returns>
        public static string Protect(string str)
        {
            // Use assembly full name as entropy
            byte[] entropy =
                Encoding.ASCII.GetBytes(Assembly.GetExecutingAssembly().FullName);
            byte[] data = Encoding.ASCII.GetBytes(str);
            string protectedData = Convert.ToBase64String(ProtectedData.Protect(data,
                entropy, DataProtectionScope.CurrentUser));
            return protectedData;
        }

        /// <summary>
```

המחלקה מכילה בתוכה אובייקט Notes

המחלקה מייצגת תא בודד בלוח הסודוקו ומשמשת כבסיס לניהול הלוגיקה והאינטראקציה עם תא הסודוקו בUI. היא יורשת מהמחלקה TextBox ומרחיבה אותה בכמה דרכים משמעותיות.

תפקידים עיקריים של המחלקה Cell

אתחול והגדרות תא: קביעת המיקום של התא בלוח הסודוקו, אתחול תכונות חזותיות כמו גודל הפונט, צבעי רקע וגבול, וקביעת הערך הנכון של התא בפתרון הסודוקו.

ניהול אירועים: טיפול באירועים שונים כמו שינוי טקסט, קלט טקסט, לחיצה על מקשים ומעבר פוקוס, על מנת לספק חוויית משתמש חלקה ואינטראקטיבית.

התאמת חזותית: התאמת גודל הפונט של הטקסט בתא בהתאם לגודל התא ולתוכן המוכנס, כדי לשמור על נראות טובה בכל הגדלים.

ניהול והצגת הערות: אפשרות להוסיף ולהציג הערות בתוך התא, המאפשרות לשחקן לרשום הערות קטנות שיסייעו לו בפתרון הפאזל.

בדיקת תקינות: בדיקת הקלט שנכנס לתא ובדיקת תקינות התוכן בתא יחסית לתאים אחרים בלוח, כדי להבטיח שהמשחק יתקדם בצורה נכונה ותקינה.

קוד מחלקת Cell :

```
using Sudoku_WPF.publico;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Input;
using System.Windows.Media;
using static Sudoku_WPF.publico.Constants;

namespace Sudoku_WPF.GameClasses
{
    > /// summary<
    /// Represents a single cell in the Sudoku board, derived from TextBox.
    /> /// summary<
```

המחלקה GamelInfo אחראית לאחסון מידע על משחק סודוקו מסוים. מידע זה נועד לשמש לניהול ושמירת מצב המשחק ולצורך המשך משחק שמור. היא מכילה נתונים התואמים את הטורים במסד הנתונים.

קוד מחלקת: GamelInfo

```
using static Sudoku_WPF.publico.Constants;
using Sudoku_WPF.GameClasses;
using System.Windows.Controls;
using System.Windows.Threading;
using Sudoku_WPF.publico;

namespace Sudoku_WPF.GameClasses
{
    public class GameInfo
    {
        public string BoardCode { get; set; }
        public int BoxHeight { get; set; }
        public int BoxWidth { get; set; }
        public string PuzzleCode { get; set; }
        public string Name { get; set; }
        public string Time { get; set; }
        public int Hints { get; set; }
        public int Checks { get; set; }
        public string Date { get; set; }
        public bool Solved { get; set; }
        public bool Current { get; set; }
        public DifficultyLevel DifficultyLevel { get; set; }

        public int Id { get; set; }

        public GameInfo(int id, string name, string boardCode, string puzzleCode, string
time, int hintsTaken, int checksTaken, string date, DifficultyLevel difficultyLevel, bool
solved, bool current, int boxHeight, int boxWidth)
        {
            Name = name;
            BoardCode = boardCode;
            PuzzleCode = puzzleCode;
            Time = time;
            Hints = hintsTaken;
            Checks = checksTaken;
            Date = date;
            Solved = solved;
            Current = current;
            Id = id;
            BoxHeight = boxHeight;
            BoxWidth = boxWidth;
            DifficultyLevel = difficultyLevel;
        }
    }
}
```

המחלקה Notes נועדה לניהול של הערות במשחק סודוקו. המחלקה יורשת מ- UniformGrid ומספקת לשחקנים יכולות להוסיף, להסיר ולנקות הערות על לוח המשחק.

תפקידים עיקריים של המחלקה Notes:

אתחול והגדרות ההערות:

יצירת רשימה של TextBlock לייצוג ההערות.

הגדרת ההערות באס על ידי שימוש ב- UniformGrid.

ניהול הערות:

הוספה והסרה של הערות וניקוי כללי שלהן.

פריסת הערות:

התאמת מספר השורות והעמודות בUniformGrid בהתאם למספר ההערות הקיימות, כך שהפריסה

תהיה אסתטית ונוחה לקריאה.

קוד מחלקת Notes :

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Controls.Primitives;
using System.Windows.Controls;
using System.Windows;
using System.Windows.Media;
using Sudoku_WPF.publico;

namespace Sudoku_WPF.GameClasses
{
    /// <summary>
    /// Represents a grid of notes for a Sudoku game.
    /// This class provides methods to manipulate the notes, including adding,
    removing, and clearing notes.
    /// </summary>
    public class Notes : UniformGrid
    {
        /// <summary>
        /// List of TextBlock elements representing the notes.
        /// </summary>
        public List<TextBlock> notes;
```

המחלקה Game אחראית לניהול משחק. היא כוללת את הלוח, הטיימר, והפאזל, ומנהלת את ההתקדמות והסיום של המשחק.

תפקידי המחלקה Game

אתחול משחק: המחלקה מאפשרת התחלת משחק סודוקו חדש עם לוח משחק חדש או ממידע משחק שמור.

ניהול טיימר: המחלקה מטפלת בטיימר של המשחק, מעדכנת את הזמן שחלף ומציגה אותו.

סיום המשחק: המחלקה מטפלת בסיום המשחק, כולל שמירה על המידע המשחק וסיום הטיימר.

אירועים: המחלקה מאזינה ומגיבה לאירוע פתרון המשחק.

קוד למחלקת Game:

```
using System;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Input;
using System.Windows.Threading;
using static Sudoku_WPF.publico.Constants;

namespace Sudoku_WPF.GameClasses
{
    /// <summary>
    /// Represents a Sudoku game session.
    /// </summary>
    public class Game
    {
        private Board board;
        private TextBlock timerTxtB;
        private DispatcherTimer timer;
        private TimeSpan elapsedTime;
        private Puzzle puzzle;
        private bool inProgress;

        /// <summary>
        /// Initializes a new instance of the <see cref="Game"/> class for starting
        a new Sudoku game.
        /// </summary>
        /// <param name="sodukoGrid">The grid where the Sudoku game is
        displayed.</param>
        /// <param name="timerTxtB">The text block for displaying the game
        timer.</param>
        public Game(Grid sodukoGrid, TextBlock timerTxtB)
```

מחלקת פאזל -API (puzzle) המחלקה Puzzle מייצגת פאזל סודוקו.

ייצוג פאזל : המחלקה מכילה משתנים לשמירה על הפאזל פתור (solvedPuzzle) ואילו תאים לא ניתנים לשינוי (initialCells). זה מאפשר את הלוגיקה של חידת הסודוקו (הלוח במצבו ההתחלתי).

ייצירת פאזל:

יש למחלקה אפשרות ליצור פאזל חדש או מקוד פאזל קיים.

פאזל חדש נוצר באמצעות יצירת לוח פתור ב BachTrecking באמצעות רקורסיה (ראה דברים מעניינים בפרויקט).

וקוד פאזל קיים מחזיק מידע על הפאזל הפתור ועל התאים ההתחלתיים בקוד בפרוטוקול שפיתחתי. הקוד מגיע מוצפן ([ראה מחלקת Code](#)).

פעולות על תאים: המחלקה מציעה שיטות לבדיקה האם תא מסוים הוא תא התחלתי, ולקבלת ערך של תא בפאזל הפתור.

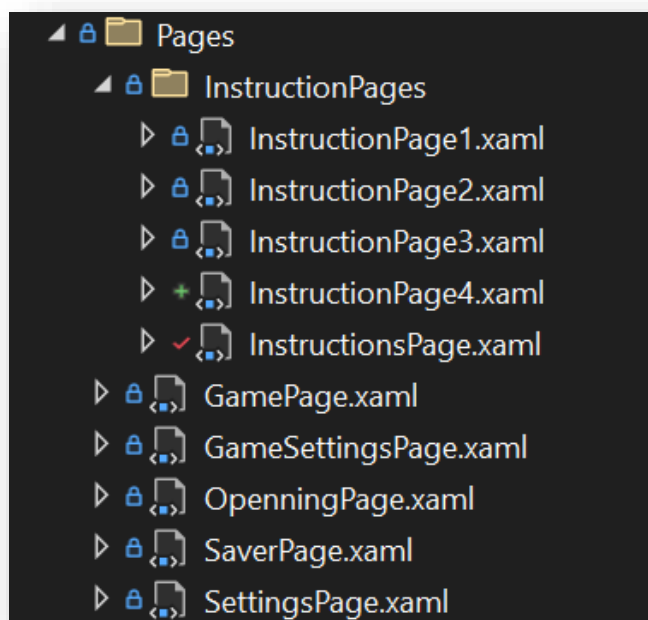
ניהול קוד פאזל: ישנם שיטות שמייצגות את הפאזל באמצעות קוד, ומאפשרות ייבוא ויצוא של מצב הפאזל בין מצבים שונים.

הקוד למחלקת Puzzle:

```
using System;
using Sudoku_WPF.publico;
using static Sudoku_WPF.publico.Constants;

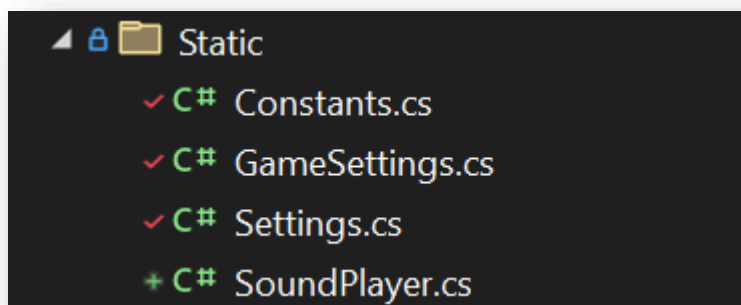
namespace Sudoku_WPF.GameClasses
{
    /// <summary>
    /// Represents a Sudoku puzzle and provides methods for puzzle generation, management, and access.
    /// </summary>
    public class Puzzle
    {
        public static bool[,] initialCells; // Array to track which cells are initial (pre-filled)
        public static char[,] solvedPuzzle; // Array to store the solved puzzle
        public string code = "NO CODE YET"; // Code representing the puzzle state
        private static Random rnd = new Random(); // Random number generator for shuffling and selecting initial cells
```

התיקיה מכילה את כל העמודים
המשומשים בפרוייקט ראה את
העמודים, את תפקידם ואת קודם ראה
ב "[דפים](#)". ראה שיטת ניווט ב "[מפת
הדפים](#)"



מחלקות סטטיות

תיקיה המכילה את המחלקות
הסטטיות שמטרתן להחזיק מידע
מוכר לכולם במהלך ריצת התכנית.



Constants – קובץ המכיל מחלקת קבועים שבתוכה תת מחלקות של קבועים.

SoundPlayer - מחלקה שמטפלת בענייני שמע כמו צלילים ומוזיקה.

GameSettings – מחלקה השומרת את הגדרות המשחק הנוכחי

Settings – מחלקה השומרת את ההגדרות הנוכחיות

דפים

דף בית



העמוד מכיל לוגו גדול של סודוקו למטרות יופי שעולה ויורד בלופ בעזרת (double animation).

מטרתו לקבוע את הגדרות המשחק:

העמוד מאפשר יצירת משחק בשתי דרכים, יצירת משחק חדש בגודל הנקבע לפי ה NumericUpDown ורמת קושי הנקבעת בערך המוכנס ב Combo box.

הוא מאפשר גם הדבקת קוד פאזל (אפשר לשלוח בווצאפ) להתחלת משחק שנוצר בעבר מחדש. הקוד מוצפן לכן אין כל סכנה שבהדבקה המשתמש ישיג מידע על פתרון הלוח.

```

using ControllLib;
using Sudoku_WPF.GameClasses;
using Sudoku_WPF.publico;
using Sudoku_WPF.Themes;
using System;
using System.Collections.Generic;
using System.Diagnostics.Metrics;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Data;
using System.Windows.Documents;
using System.Windows.Input;
using System.Windows.Media;
using System.Windows.Media.Imaging;
using System.Windows.Navigation;
using System.Windows.Shapes;
using static Sudoku_WPF.publico.Constants;

namespace Sudoku_WPF
{
    /// <summary>
    /// Interaction logic for SettingsPage.xaml
    /// </summary>
    public partial class GameSettingsPage : Page
    {
        public Page gamePage;

        /// <summary>
        /// Constructor for initializing the GameSettingsPage.
        /// Sets initial theme, max values for number up down controls, and event
handlers.
        /// </summary>
        public GameSettingsPage()
        {
            InitializeComponent();

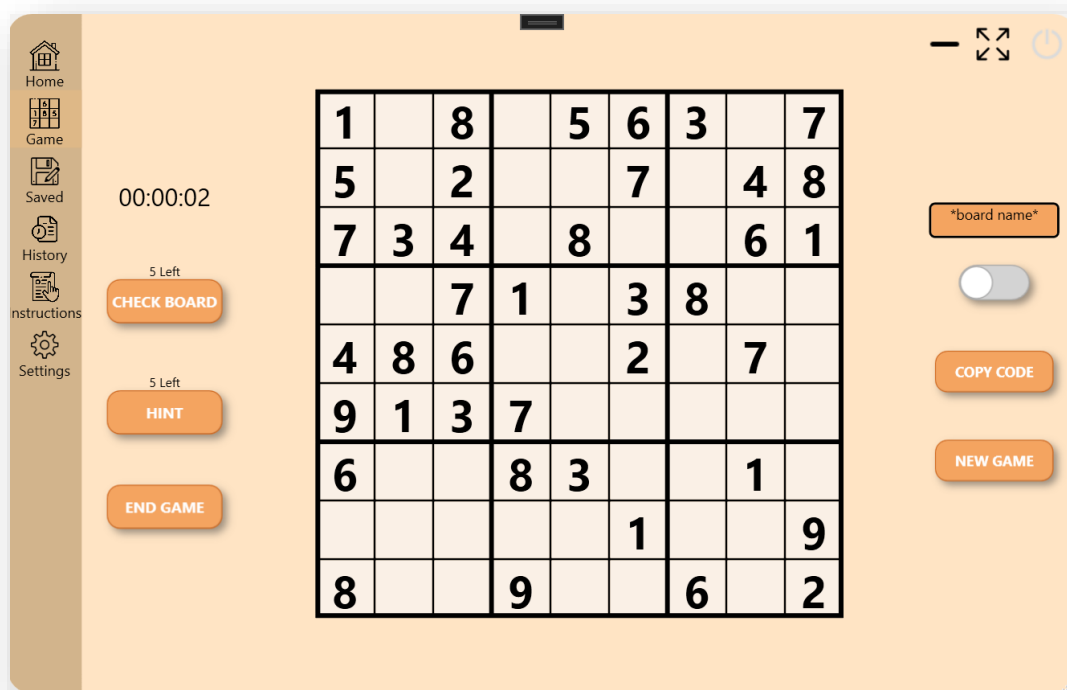
            SetInitialTheme(GameSettings.difficultyLevel.ToString());

            NUD_boxHeight.Value = GameSettings.BoxHeight;
            NUD_boxWidth.Value = GameSettings.BoxWidth;

            gamePage = null;
        }

        /// <summary>
        /// Sets the initial theme based on the provided theme name.
        /// </summary>
        /// <param name="themeName">The name of the theme to set.</param>
        private void SetInitialTheme(string themeName)
        {
            // Find the ComboBoxItem with the specified theme name and set it as
selected
            var selectedItem = DifficultyLevel_CMBB.Items
                .OfType<ComboBoxItem>()
                .FirstOrDefault(item => item.Content.ToString() ==
themeName);

```



עמוד משחק בו משחקים. מאפשר:

הכנסת שם לוח

עצירה זמנית של המשחק

העתקה אוטומטית של קוד

יצירת משחק חדש (ושמירת הלוח הזה)

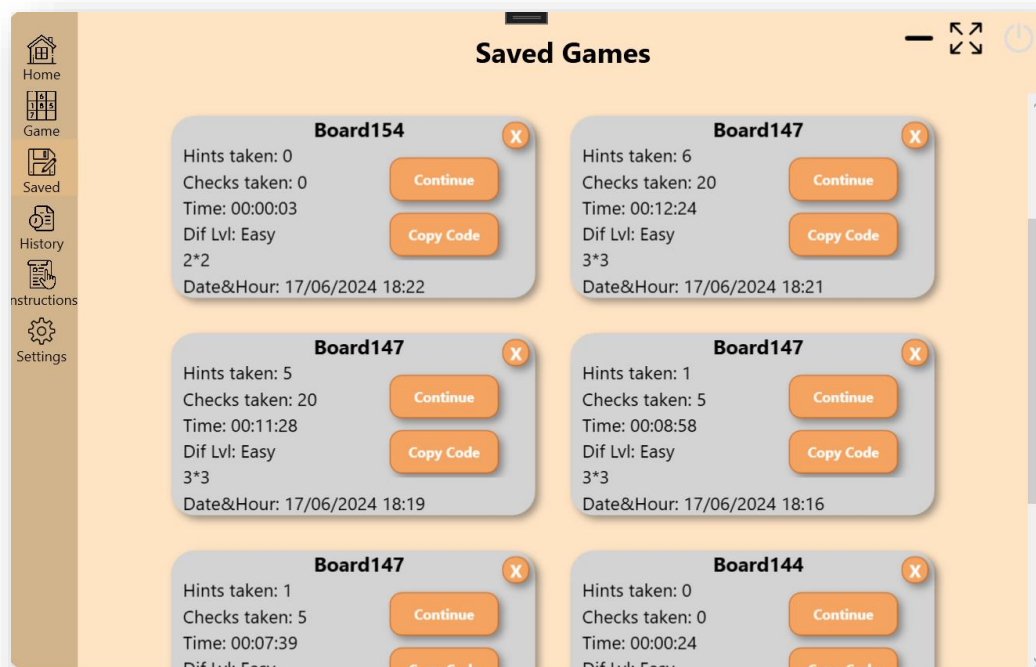
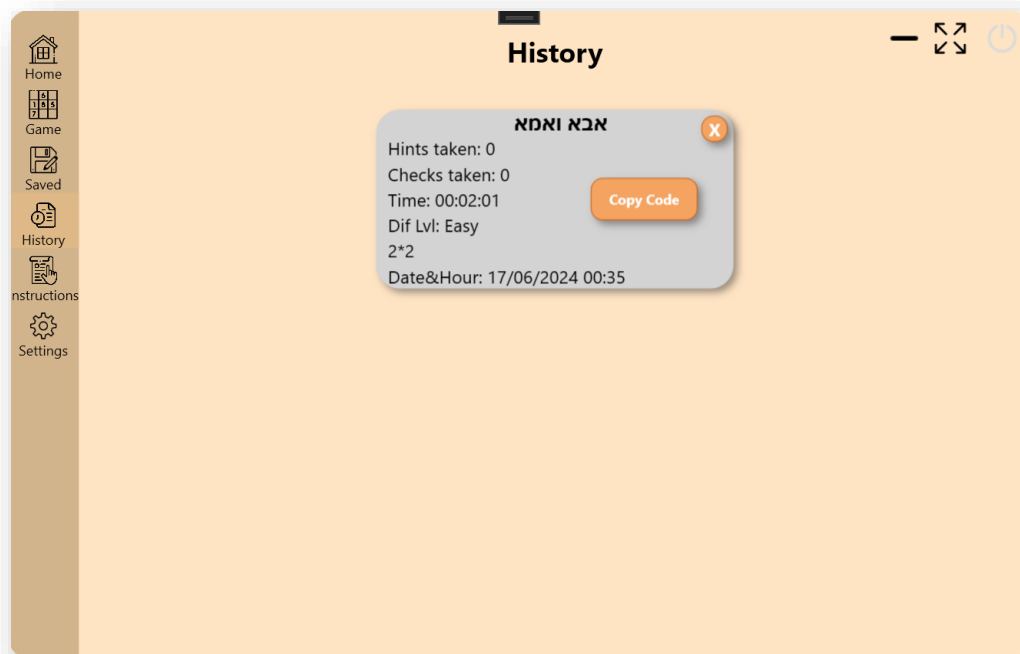
סיום לוח והצגת הפתרון (ושמירת הלוח הזה)

תצוגת הזמן שעבר עד כה

בדיקת לוח

רמז (קבלת פתרון המשבצת בפוקוס)

דף הסטוריה ודף משחקים שמורים



העמודים מציגים משחקים שנשמרו (בזיכרון של התכנה ובמסד הנתונים).
עמוד ההסטוריה מראה את המידע על המשחקים שפתרו בהצלחה ואפשרות להעתיק את קודם.

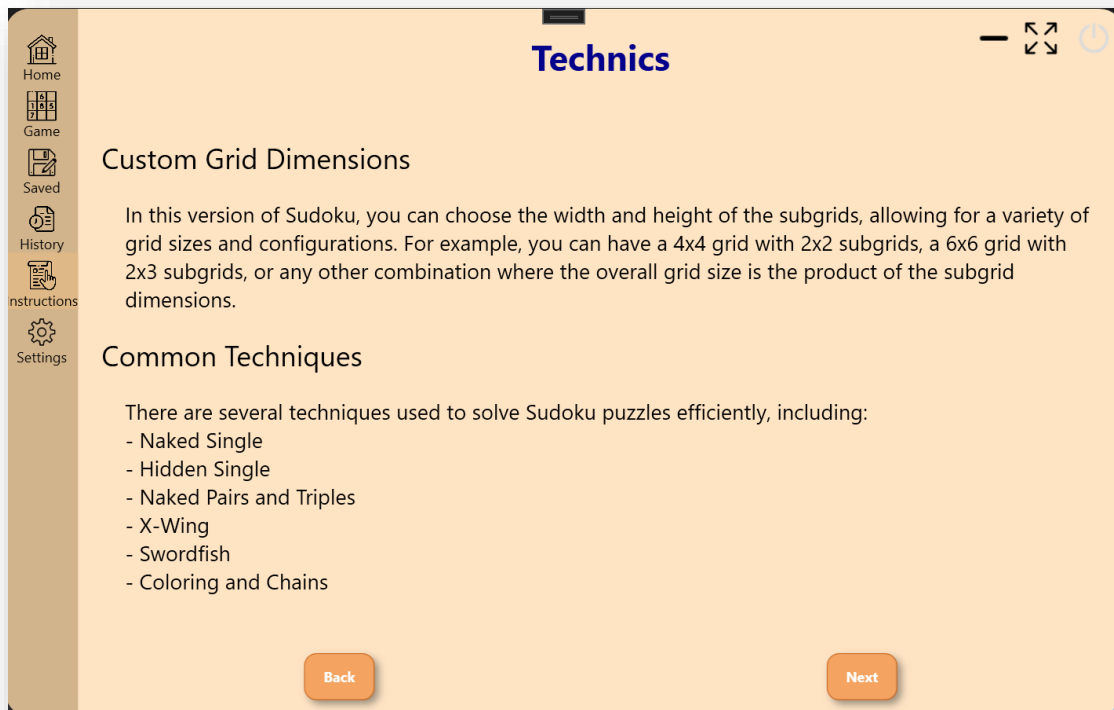
עמוד המשחקים השמורים מציג את המידע על המשחקים השמורים ומאפשר להמשיכם, למחוקם ולהעתיק קודם.

עמוד ההסטוריה והשמירה הם מופעים שונים של אותו עצם (עמוד) ולהם אותו קוד, לכן אקרא להם "עמוד שומר".

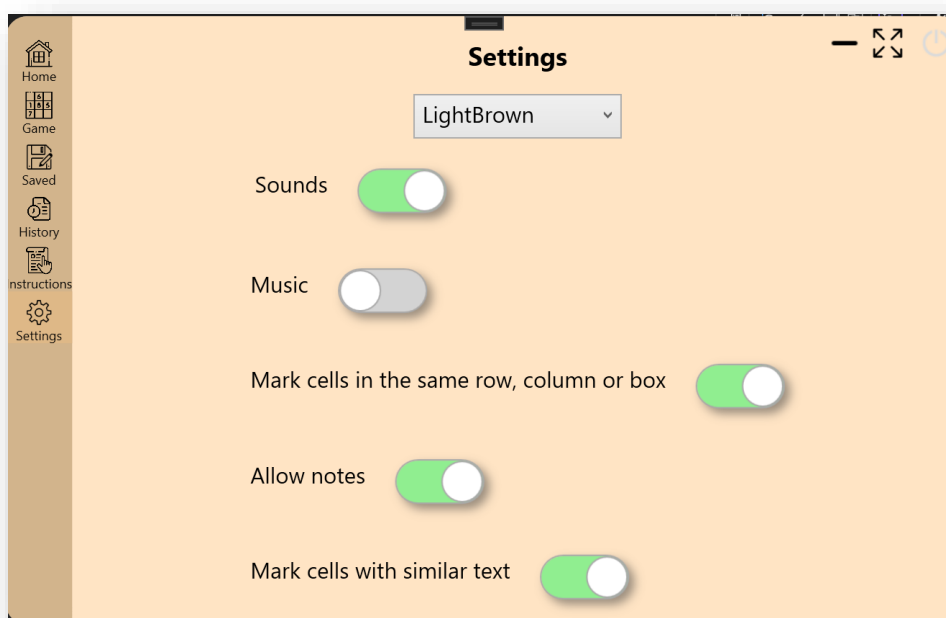
דף הצגת הוראות



מאפשר מעבר בין עמודי הוראות וקריאתם.



מכיל את תוכן ההערות בלבד. הכפתורים חלק מעמוד הצגת ההוראות



עמוד הגדרות המאפשר הגדרות תכנה ושינוי מצב הצבע בכל התכנה.

אלגוריתמים ויכולות מיוחדות

יצירת לוח פתור

קריאה לפונקציה הרקורסיבית במקום הראשון בלוח

```
/// <summary>
/// Generates a solved Sudoku puzzle based on the current state of the board.
/// </summary>
1 reference
private void GenerateSolvedPuzzle()
{
    FillCells(0, 0);
}
```

סיום בסוף הלוח

```
if (row == GameSettings.BoardSide - 1 && col == GameSettings.BoardSide)
    return true;
```

ירידת שורה או מעבר לתא הבא במקרה הצורך

```
if (col == GameSettings.BoardSide)
{
    row++;
    col = 0;
}
if (solvedPuzzle[row, col] != '\0')
    return FillCells(row, col + 1);
```

בדיקת איזה מספר אפשר להכניס בתא כך שהלוח ישאר נכון, והאם בכלל אפשר

```
char[] nums = GenerateShuffledNumbers();

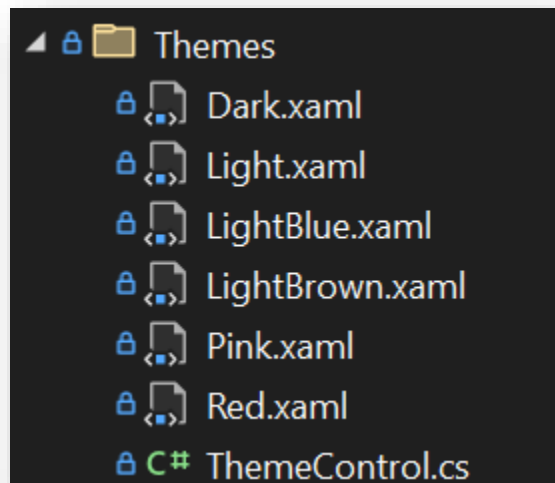
foreach (char num in nums)
{
    if (IsSafeToPlace(row, col, num))
    {
        solvedPuzzle[row, col] = num;
        if (FillCells(row, col + 1))
            return true;
        solvedPuzzle[row, col] = '\0';
    }
}

return false;
```

אם מחזיר false התא נשאר null וכאן בעצם הולכים אחורה חזרה ב Back Tracking

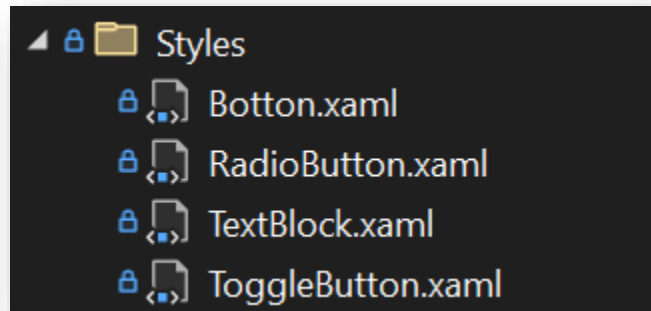
Color themes

התכנה מחליפה את נושא הצבע שלה בצורה ריספונסיבית עזרת רפרנסים ל dynamic resources של Themes ה Themes הם resource dictionaries המכילים את שמות הצבעים כמפתחות ואת הצבעים ב RGB כערכים.



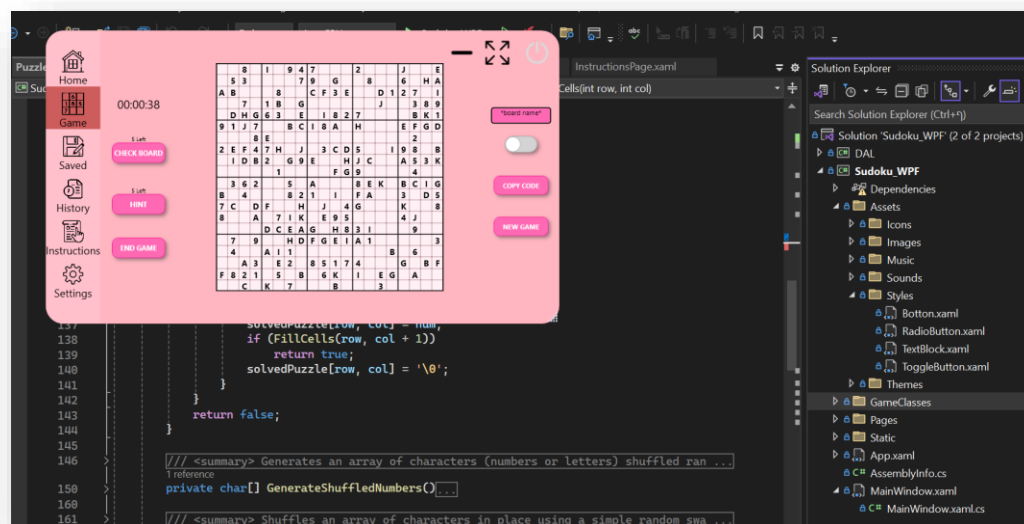
חזרה על קוד עיצובי

בתכנה אין הרבה חזרה על קוד עיצובי משום שלמדתי להשתמש ב styles לכפתורים מסודים שונים:



רספונסיביות ודינמיות מסך

חלון האפליקציה דינאמי כמעט חלוטין, הוא מאפשר שינוי גודל לכל הכיוונים עם עיווט מינימלי של התכנה. עשיתי זאת ללא ViewBox על כל החלון כי זה לא התאפשר בשל שקיפות החלון עצמו.



רכיבים חיצוניים

צירפתי חבילת Nuget בשם [System.Data.OleDb.8.0.0](#)

כדי לטפל בבאג שהיה לי עם הגישה לדאטה בייסים.

צירפתי גם חבילת Nuget בשם [NumericUpDown](#)

שמספקת אובייקת לבחירת ערך מספרי בצורה נוחה (ראה עמוד הגדרות משחק)

רפלקציה

בעיות במהלך הדרך

- היו לי אינספור בעיות ושגיאות במהלך הדרך ועל כולן הצלחתי להתגבר. המרכזיות שבהן היו:
- התקשיתי בחיבור הפרויקט DAL לפרויקט שלי – הפתרון היה התקנת [Nugget Package](#).
- לקח לי זמן רב להשיג את האלגוריתם ליצירת הלוחות הפתורים.
- התקשיתי בהפיכת חלון האפליקציה לריספונסיבי ועלו בעיות תצוגה רבות עד שהגעתי לתוצאה הסופית.
- נושאי הצבעים לא עבדו כראוי במהלך משחק כי תחילה ניסיתי לקבוע אותם דרך משתנים בCS המחזיקים את ערכי הצבעים – הפתרון היה לקשור את תכונת האובייקט לresource reference עם השם המתאים כדי להבטיח שינוי צבע מיידי ורספונסיבי.

דברים שלמדתי

במהלך ביצוע פרויקט הסודוקו, למדתי המון נושאים חשובים שהעשירו את הידע שלי בתחום התכנות והפיתוח. למדתי כיצד ליישם רקורסיה כדי לפתור בעיות מתמטיות כמו הפיתרון של לוחות הסודוקו בצורה יעילה ומובנית. גם למדתי על ירושה וכיצד להשתמש בה במבנה הפרויקטים שלי, כולל הגדרת מחלקות ושימוש בהם באופן מקורי ויעיל. ניסיתי גם להבין וליישם מבני נתונים מורכבים יותר וטכניקות ניהול בסיסי נתונים, כמו שימוש ב-SQL לשאילתות לבסיסי נתונים של ACCESS ועדכונים.

בנוסף, למדתי על פיתוח תוכנה מונחה אירועים ב-WPF. כמו כן, ניסיתי לשפר את יכולות העיצוב והסגנון באמצעות יישום סטיילים ונגישות למשתמש באמצעות עיצוב והתאמת פרטים קטנים כגון שינוי גודל של חלונות.

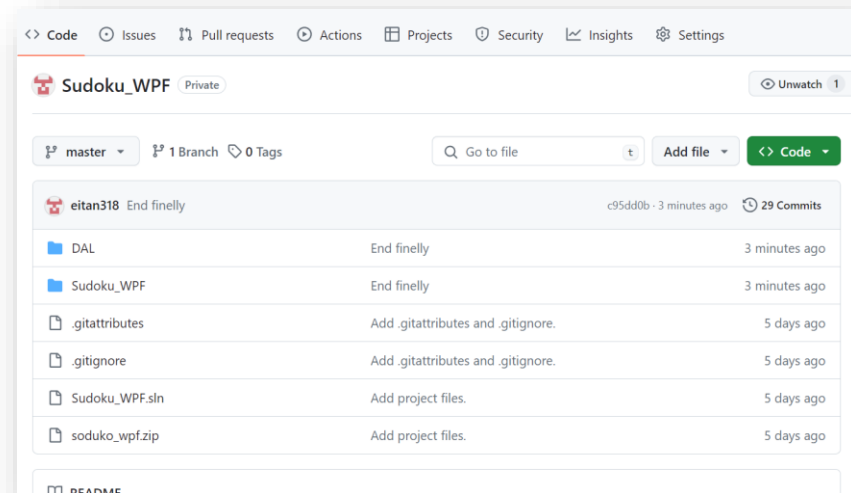
הפרויקט עזר לי להתפתח כמתכנת באופן משמעותי ולרכוש כישורים חשובים בעולם התכנות ואני בטוח שפרויקט זה הפך אותי למתכנת טוב יותר.

תהליך עבודה

בתחילת העבודה תכננתי בקפידה מה אני הולך לעשות כשהכנתי הצעת פרויקט. עבדתי על הפרויקט כמה ימים והיה לי לוח המתפקד כראוי. אז עשיתי הכל במחלקה אחת אבל כשהגיעה ל 800 שורות החלטתי שזה הולך להיות הפרויקט הראשון שלי שאממש ב OOP. לקח קצת זמן, אך לבסוף המרתי את כולו למחלקות משחק קטנות.

מאז אני עבדתי על הפרויקט מסודר – אלגוריתם אחד בכל פעם והשתדלתי לשמור על עקרונות קוד טוב: חלוקה לפונקציות, minimal API, אבסטרקציה, KISS, אנקפסולציה ופולימורפיזם.

את ניהול הגרסאות שלי ביצעתי לראשונה ב github וזה עזר לי מאוד.



לקראת תהליך ההגשה עלו לי רעיונות חדשים בכל פעם, אך החלטתי שיותר מדיי זה לא טוב ושאני צריך לממש רק את הדברים המאתגרים והחשובים ביותר.

אני מאוד שמח מהתוצאה ואני חושב שהפרויקט תרם מאוד לידע ולניסיון שלי משום שהוא הפרויקט הגדול והמורכב ביותר שעשיתי עד כה.

תוספות אפשריות

- יצירת גרסה ל/mobile
- טורנירים בין שחקנים
- חידה יומית
- רשת חברתית המאפשרת יצירת חברות סודקו ותחרויות מאורגנות